

Тема 8. Язык UML

- 1. Элементы языка UML**
- 2. Правила языка UML**
- 3. Архитектура программной системы**

UML - это

**язык для визуализации, специфицирования,
конструирования и документирования артефактов
программных систем.**

UML - это язык

UML состоит из словаря и правил, ориентированных на концептуальное и физическое представление системы.

UML – средство составления «чертежей» программного обеспечения.

UML описывает архитектуру системы с разных точек зрения.

Таким образом, словарь и правила позволяют составлять и читать «чертежи» (модели).

UML - язык визуализации

Систему можно описать словами или нарисовать графически.

Явная модель облегчает общение.

UML – не просто набор графических символов. За каждым из них стоит определённая семантика, т.е. модель написанная одним разработчиком может быть однозначно интерпретирована другими разработчиками или даже инструментальными системами.

UML - язык специфицирования

Специфицирование – построение точных, недвусмысленных и полных моделей.

UML позволяет специфицировать все решения касающиеся анализа, проектирования и реализации, которые должны приниматься в процессе разработки и развёртывания системы.

UML - язык конструирования

Модели, созданные на языке UML, можно перевести на многие языки программирования.

Прямое проектирование – генерация кода из модели UML.

Обратное проектирование – реконструкция модели по имеющейся реализации.

UML - язык документирования

- ☐ требования к системе
- ☐ архитектура
- ☐ исходный код
- ☐ проектные планы
- ☐ тесты
- ☐ прототипы
- ☐ версии и т.д.

Строительные блоки UML



Сущности

Сущности – абстракции, являющиеся основными элементами моделей.

В UML четыре типа сущностей:

- 1. Структурные**
- 2. Поведенческие**
- 3. Группирующие**
- 4. Аннотационные**

Структурные сущности

Имена существительные в моделях UML.

Представляют собой статические части моделей, соответствующие концептуальным или физическим элементам системы.

Бывает семь базовых разновидностей:

1. Классы
2. Интерфейсы
3. Кооперации
4. Прецеденты
5. Активные классы
6. Компоненты
7. Узлы

Класс (Class)

Описание совокупности объектов с общими атрибутами, операциями, отношениями и семантикой.

Window

Window
size: Area visibility: Boolean
display() hide()

Window
attributes +size: Area = (100, 100) #visibility: Boolean = true +defaultSize: Rectangle -xWin: XWindow
operations display() hide() -attachX(xWin: XWindow)

Window
attributes public size: Area = (100, 100) defaultSize: Rectangle protected visibility: Boolean = true private xWin: XWindow
operations public display() hide() private attachX(xWin: XWindow)

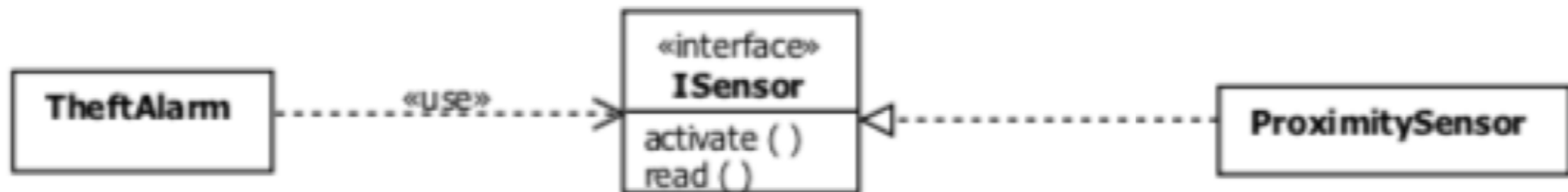
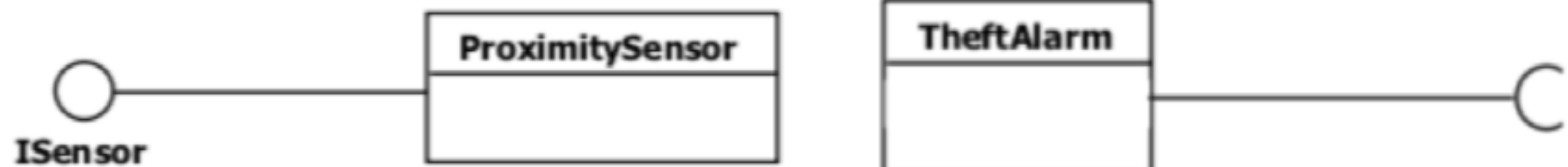
Интерфейс (Interface)

Это совокупность операций, которые определяют сервис (набор услуг), предоставляемый классом или компонентом.

Интерфейс описывает видимое извне поведение элемента.

Определяет только спецификации операций, но никогда — их реализации.

Интерфейс (Interface)



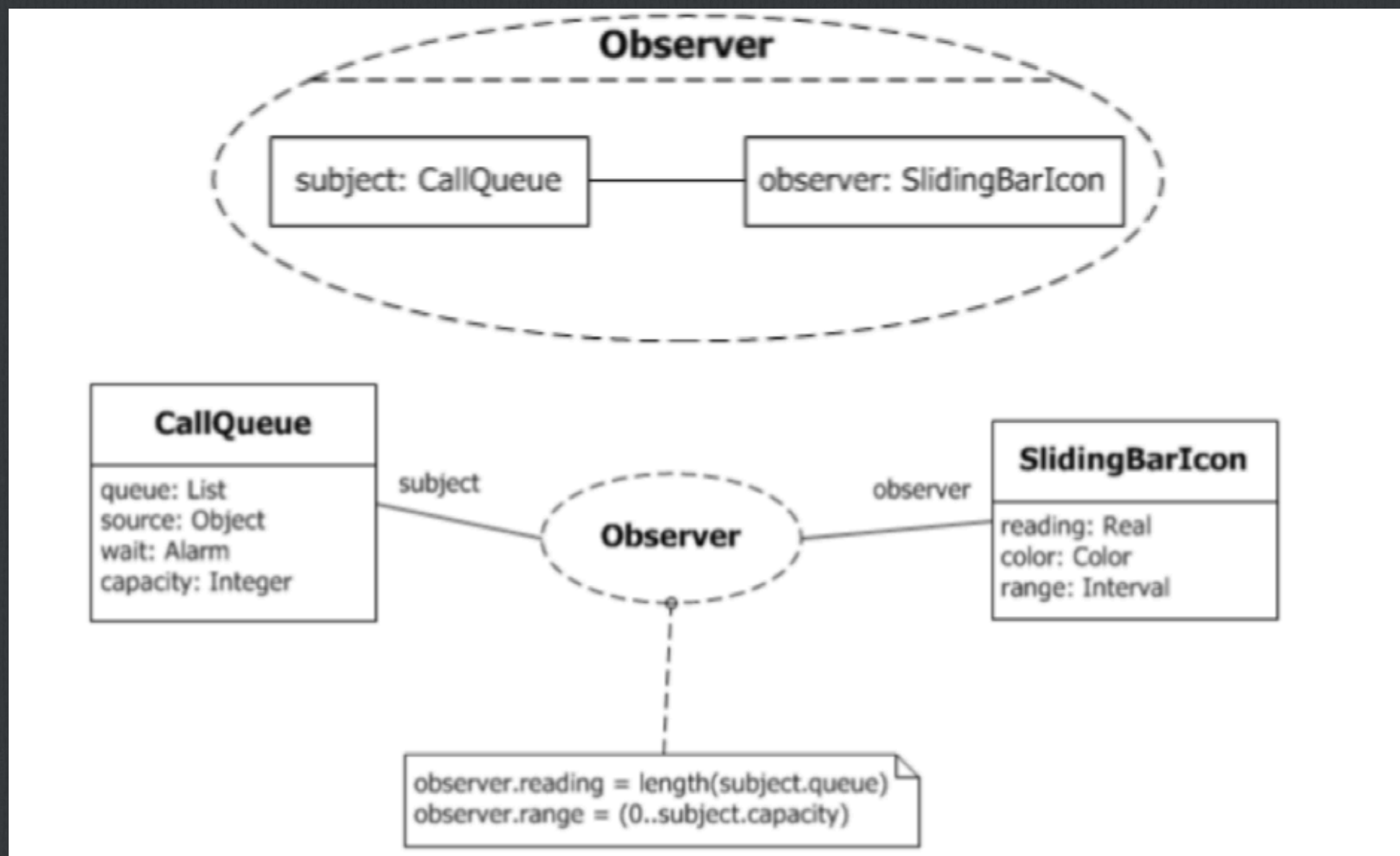
Кооперация (Cooperation)

Определяет взаимодействие.

Представляет собой совокупность ролей и других элементов, которые работая совместно, производят некоторый кооперативный эффект, не сводящийся к простой сумме слагаемых.

Кооперация включает структурную и поведенческую составляющую

Кооперация (Cooperation)

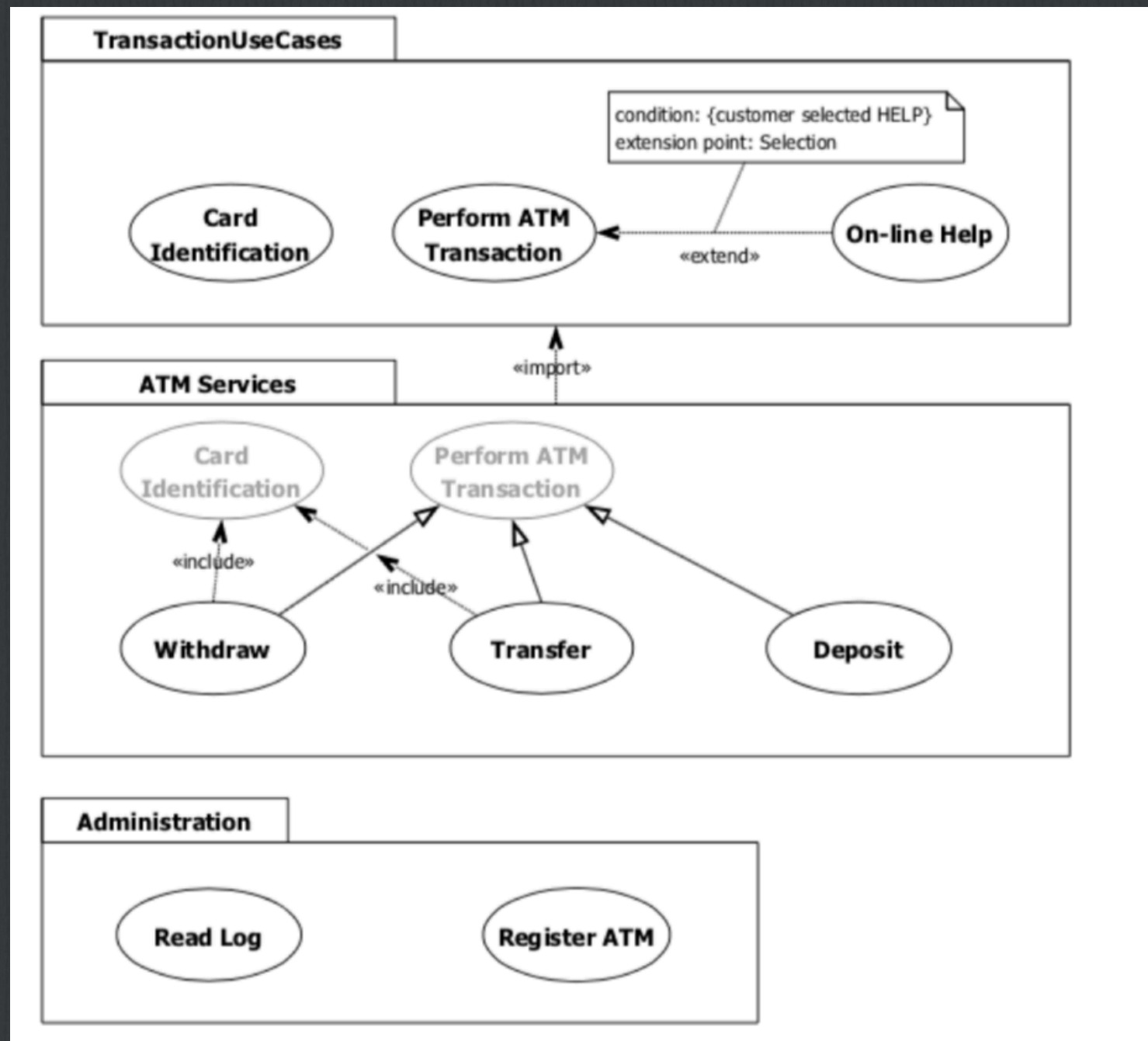


Прецедент (Use Case)

Описание последовательности выполняемых системой действий, которая производит наблюдаемый результат, значимый для определённого актёра.

Применяется для структурирования поведенческих сущностей модели. Реализуются посредством коопераций.

Прецедент (Use Case)



Активный класс (Active class)

Класс, объекты которого вовлечены в один или несколько процессов и могут инициировать управляющее воздействие.

Отличие от обычного класса – его объекты представляют собой элементы, деятельность которых осуществляется одновременно с деятельностью других элементов.

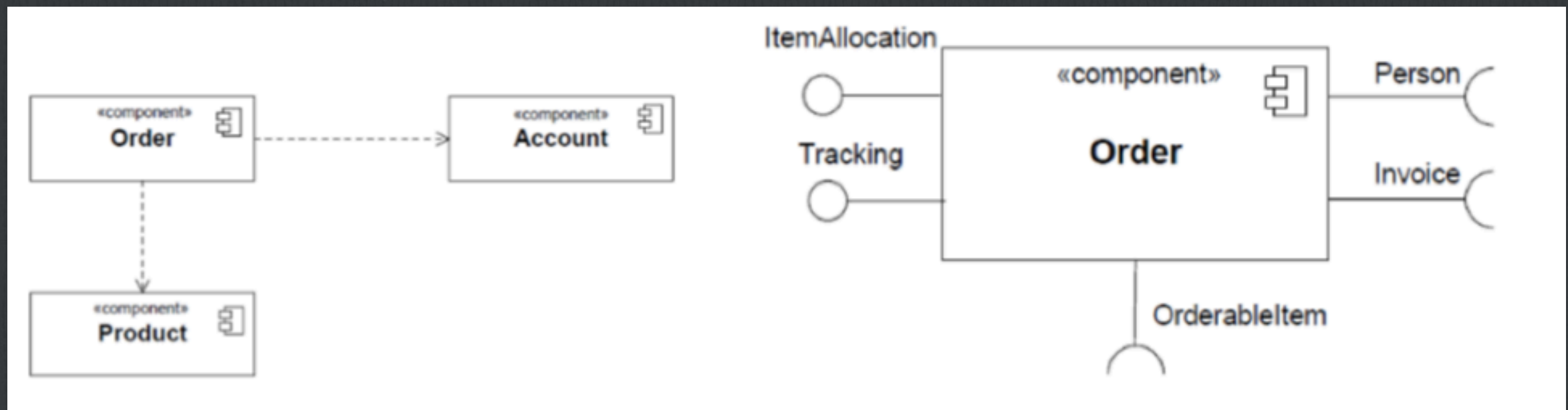


Компонент (Component)

Физическая заменяемая часть системы, которая соответствует некоторому набору интерфейсов и обеспечивает его реализацию.

Как правило представляют собой физическую упаковку логических элементов (таких, как классы, интерфейсы, кооперации)

Компонент (Component)

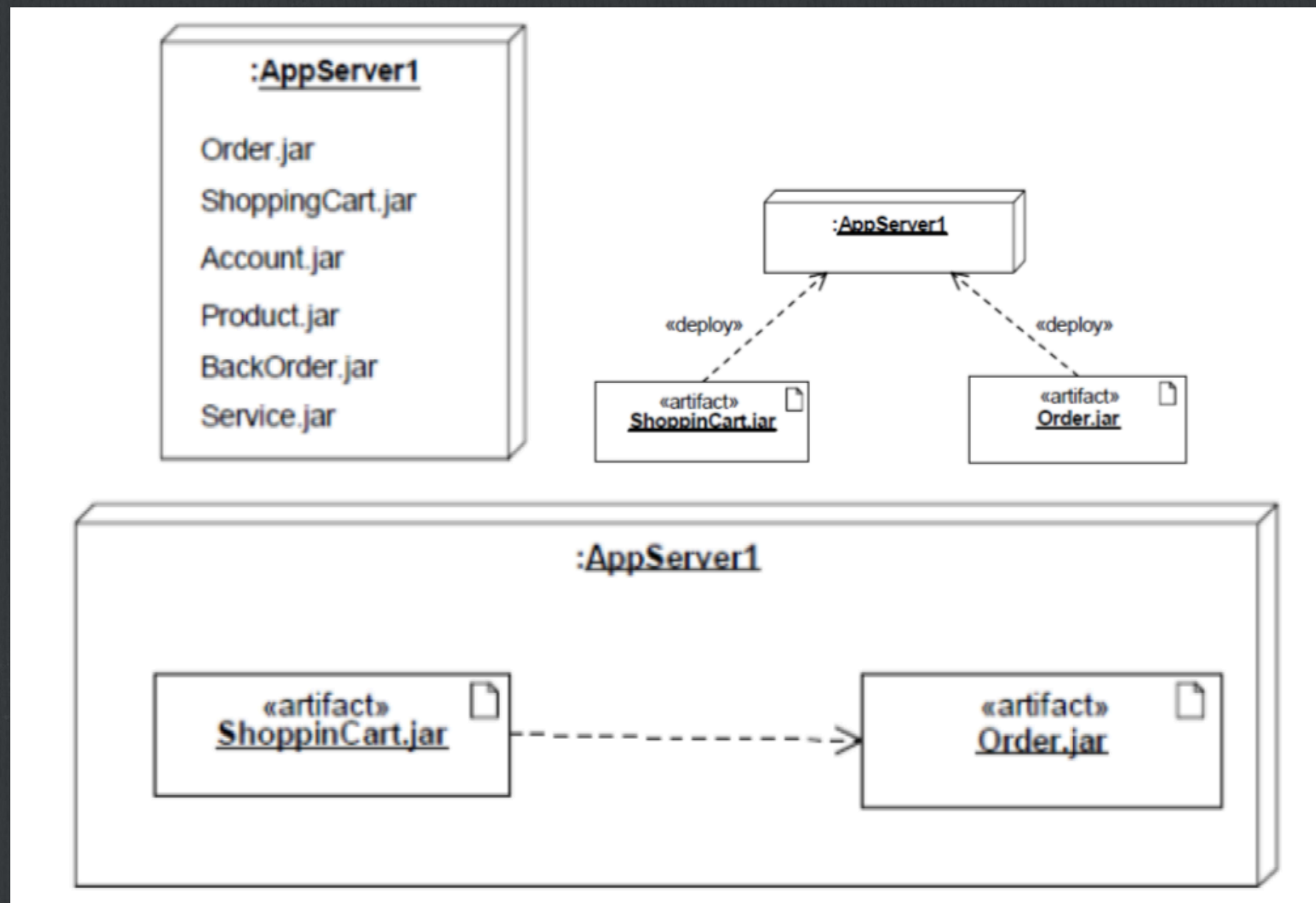


Узел (Node)

Это элемент реальной (физической) системы, который существует во время функционирования программного комплекса и представляет собой вычислительный ресурс, обычно обладающий некоторым объёмом памяти, а часто ещё и способностью обработки данных.

Совокупность компонентов может размещаться в узле, а также мигрировать с одного узла на другой.

Узел (Node)



Производные разновидности сущностей

Актёры, сигналы, утилиты – классы.

Процессы и нити – активные классы.

**Приложения, документы, файлы, библиотеки, страницы,
таблицы – компоненты.**

Поведенческие сущности

Динамические составляющие модели UML.

Это глаголы языка – они описывают поведение модели во времени и в пространстве. Для их структурирования используются прецеденты.

Существует два основных типа:

1. Взаимодействия
2. Автоматы

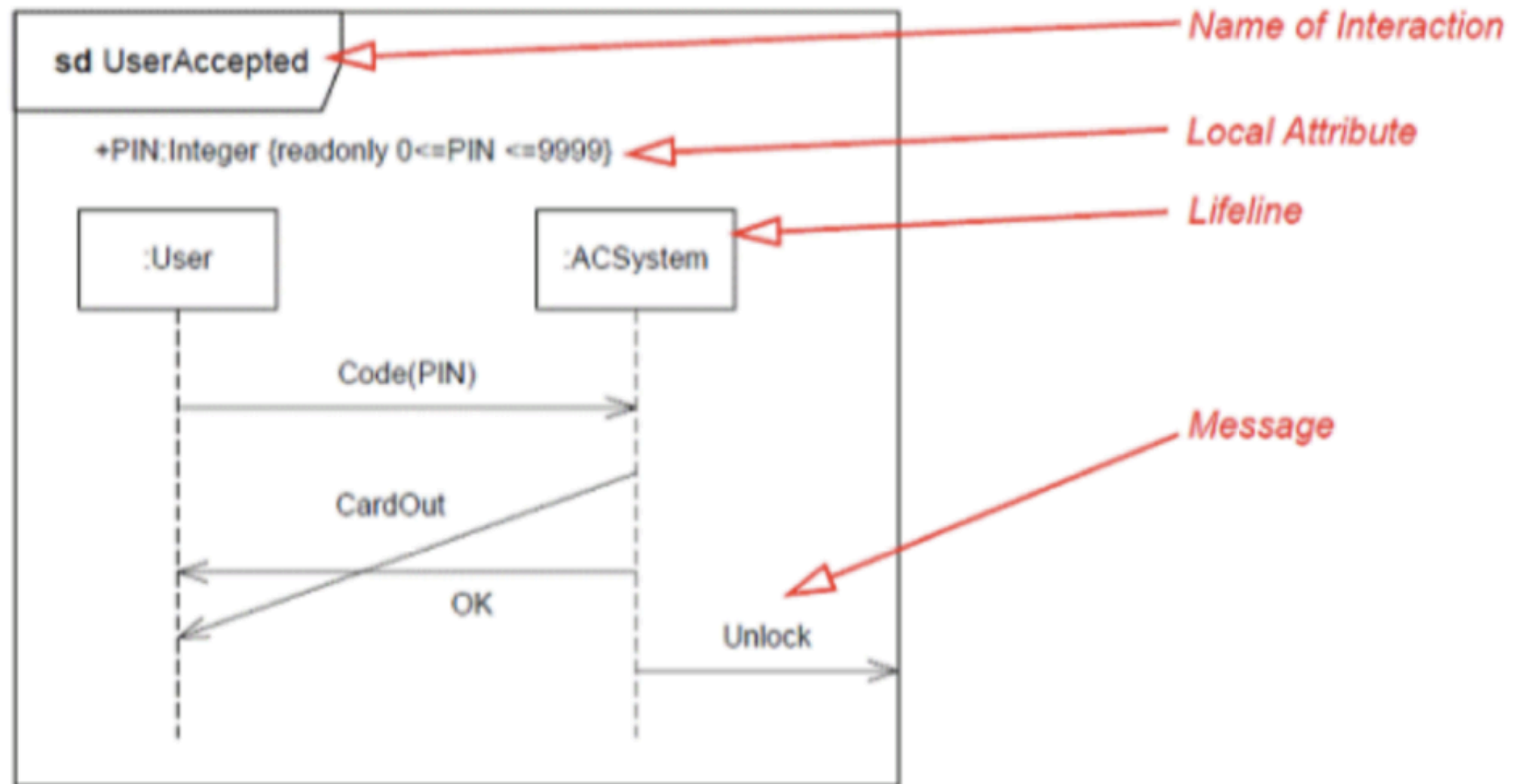
Взаимодействие (Interaction)

Поведение, сущность которого сводится к обмену сообщениями между объектами в конкретном контексте для достижения определённой цели.

Описывают как отдельную операцию, так и поведение совокупности объектов.

Взаимодействие предполагает ряд других элементов — сообщения, потоки событий, связи между объектами.

Взаимодействие (Interaction)



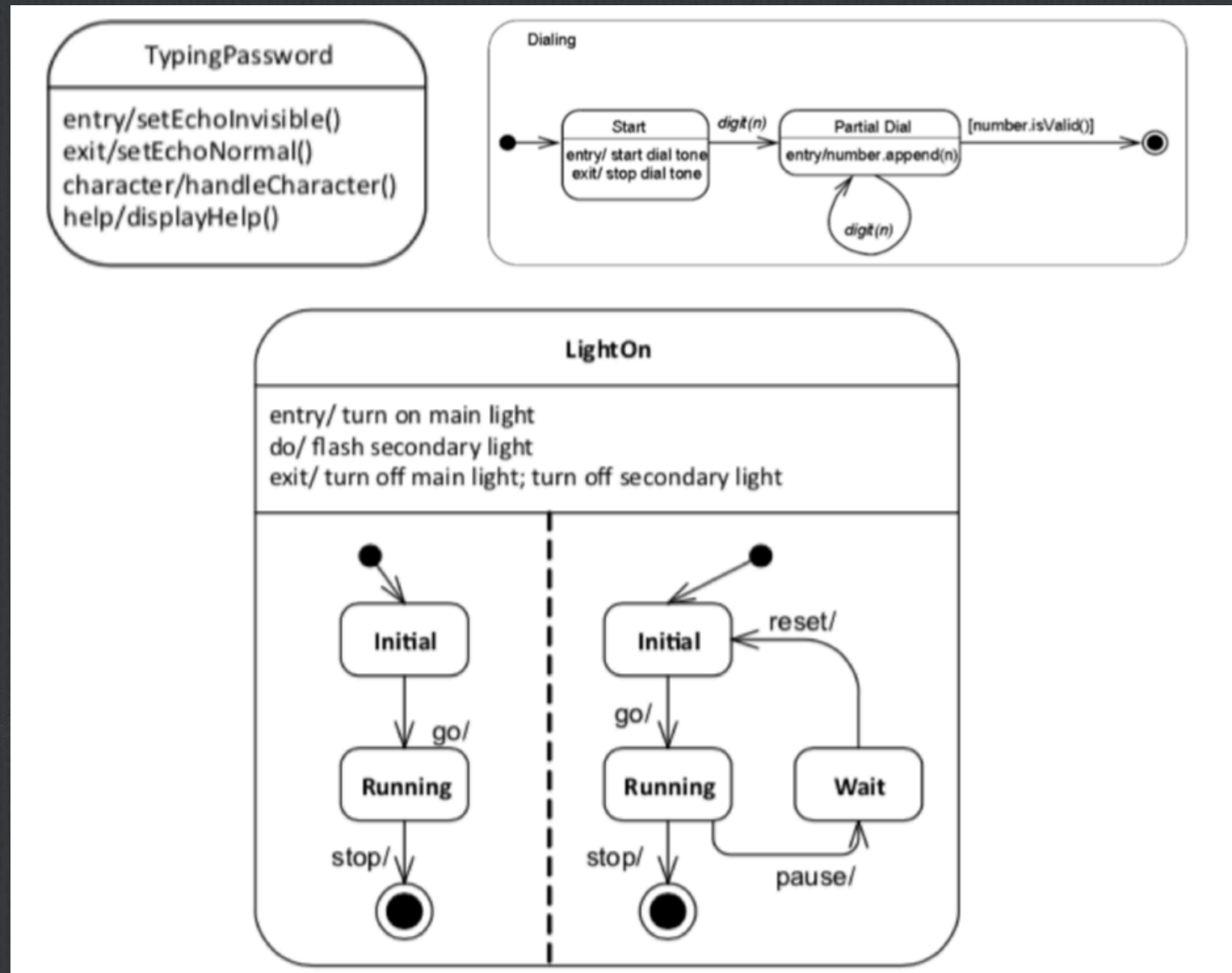
Автомат (State machine)

Алгоритм поведения, определяющий последовательность состояний, через которые объект или взаимодействие проходят на протяжении своего жизненного цикла в ответ на различные события.

С помощью автомата может быть также описано поведение класса или кооперации классов.

С автоматом связаны – состояния, переходы, события, виды действий.

АВТОМАТ (State machine)



Группирующие сущности

Это блоки, на которые можно разложить модель.

Первичная группирующая сущность – пакет.

Пакет (Package) – универсальный механизм организации элементов в группы.

Пакеты носят чисто концептуальный характер.



**Business
rules**

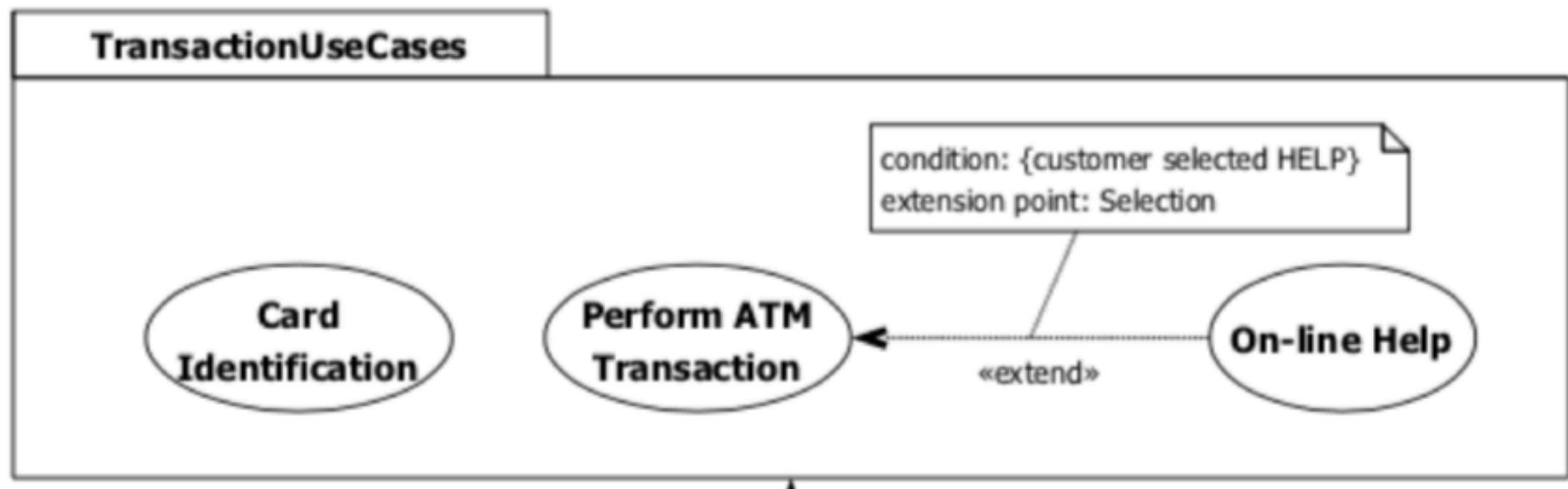
Аннотационные сущности

Это пояснительные части модели UML (дополнительные описание, замечания, разъяснения к элементам модели).

Базовый тип аннотационных элементов – примечание (note).

Will return copy of itself

Пакеты и аннотации



Отношения

Это основные связующие блоки моделей UML.

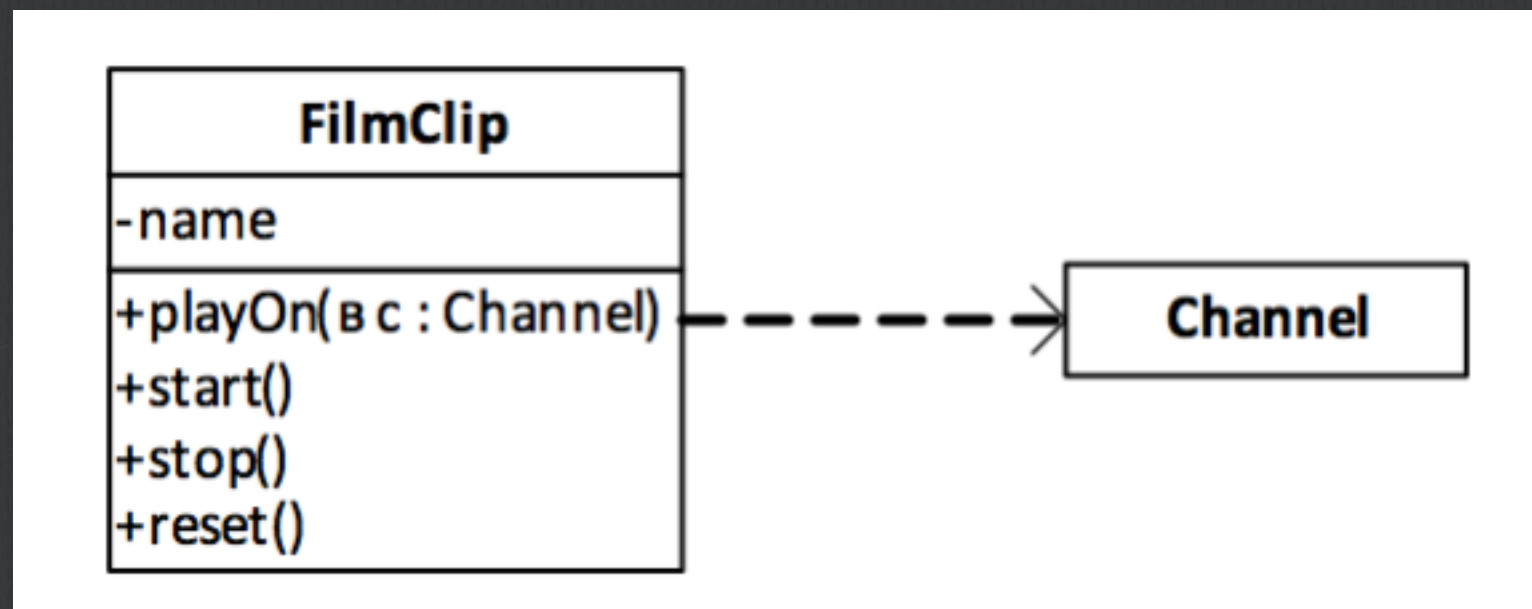
Они применяются для создания корректных моделей.

Типы отношений:

1. Зависимость
2. Ассоциация
3. Обобщение
4. Реализация

Зависимость (Dependency)

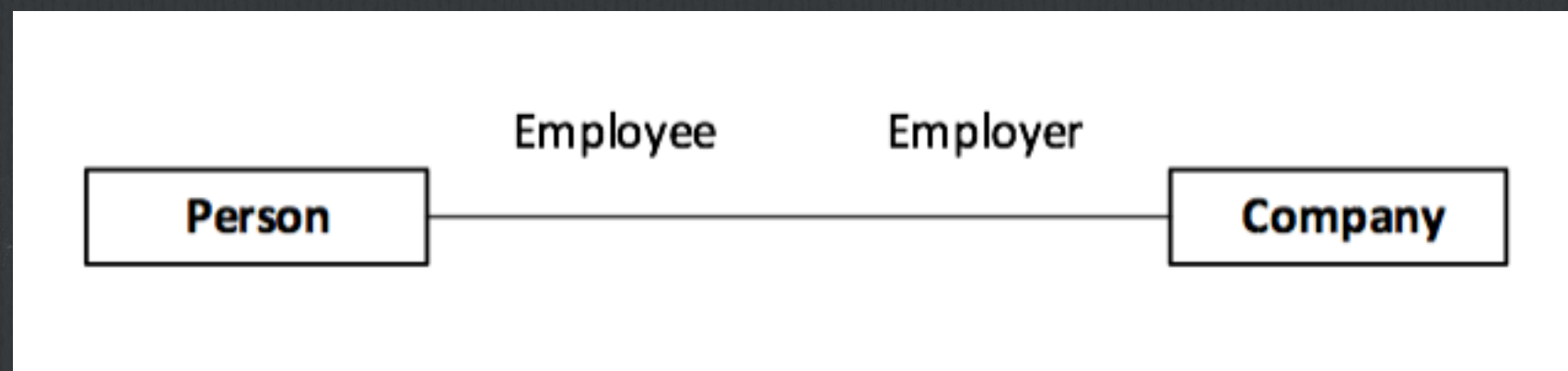
Это семантическое отношение между двумя сущностями, при котором изменение одной из них (независимой) может привести к изменению другой (зависимой).



Ассоциация (Association)

Структурное отношение, описывающее совокупность связей (соединение между объектами).

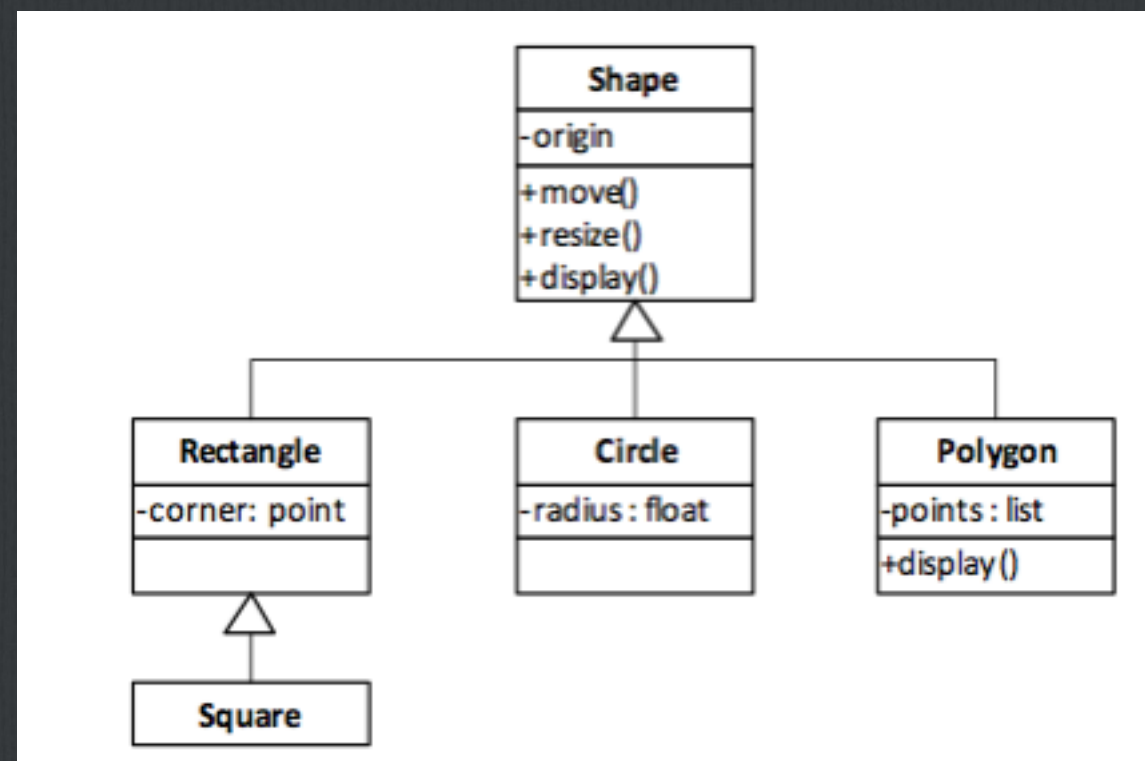
Агрегирование (aggregation) – разновидность ассоциации (структурное отношение).



Обобщение (Generalization)

Отношение между общей сущностью (родителем, суперклассом) и его конкретным воплощением (потомком, subclasses).

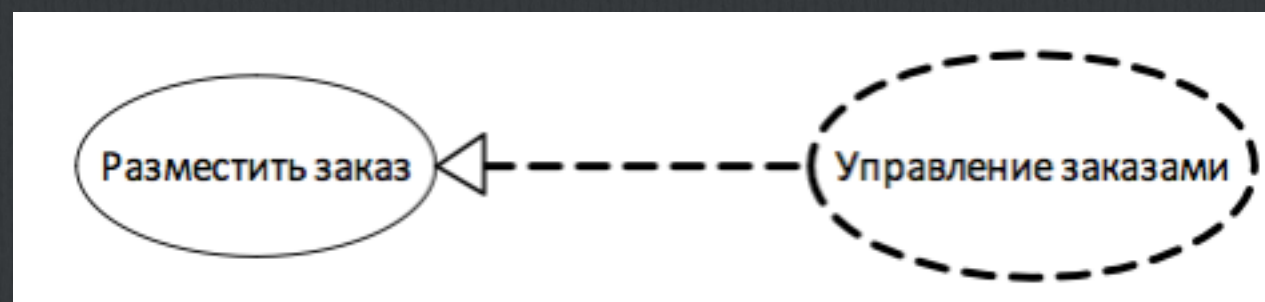
Отношение типа «является».



Реализация (Realization)

Семантическое отношение между классификаторами, при котором один классификатор определяет контракт, а другой гарантирует его выполнение.

Между интерфейсами и реализующими их компонентами, между прецедентами и реализующими их кооперациями.



Производные типы отношений

- ☐ Уточнение
- ☐ Трассировка
- ☐ Включение
- ☐ Расширение

Диаграмма UML

Графическое представление элементов модели, изображаемое в виде связанного графа с вершинами (сущностями) и ребрами (отношениями).

Рисуются для визуализации системы с различных точек зрения.

Диаграммы UML

- ☐ **Use Case** – как система будет использоваться
- ☐ **Class** – классы и связи между ними (а также объекты)
- ☐ **Activity** – активность людей или объектов
- ☐ **State machine** – различные состояния объектов со сложным жизненным циклом
- ☐ **Sequence** – то же, но с точки зрения последовательности
- ☐ **Deployment** – узлы и процессы законченной системы
- ☐ **Component** – повторно используемые компоненты и их интерфейсы

Диаграммы UML

- ☐ **Object** – объекты и связи между ними
- ☐ **Cooperation** – сообщения между объектами
- ☐ **Package** – как классы могут быть сгруппированы
- ☐ **Interaction overview** – определённые этапы деятельности диаграмм последовательности
- ☐ **Timing** – точные временные рамки для сообщений и состояний объектов
- ☐ **Composite structure** – композиции и агрегации между объектами

Диаграмма вариантов использования

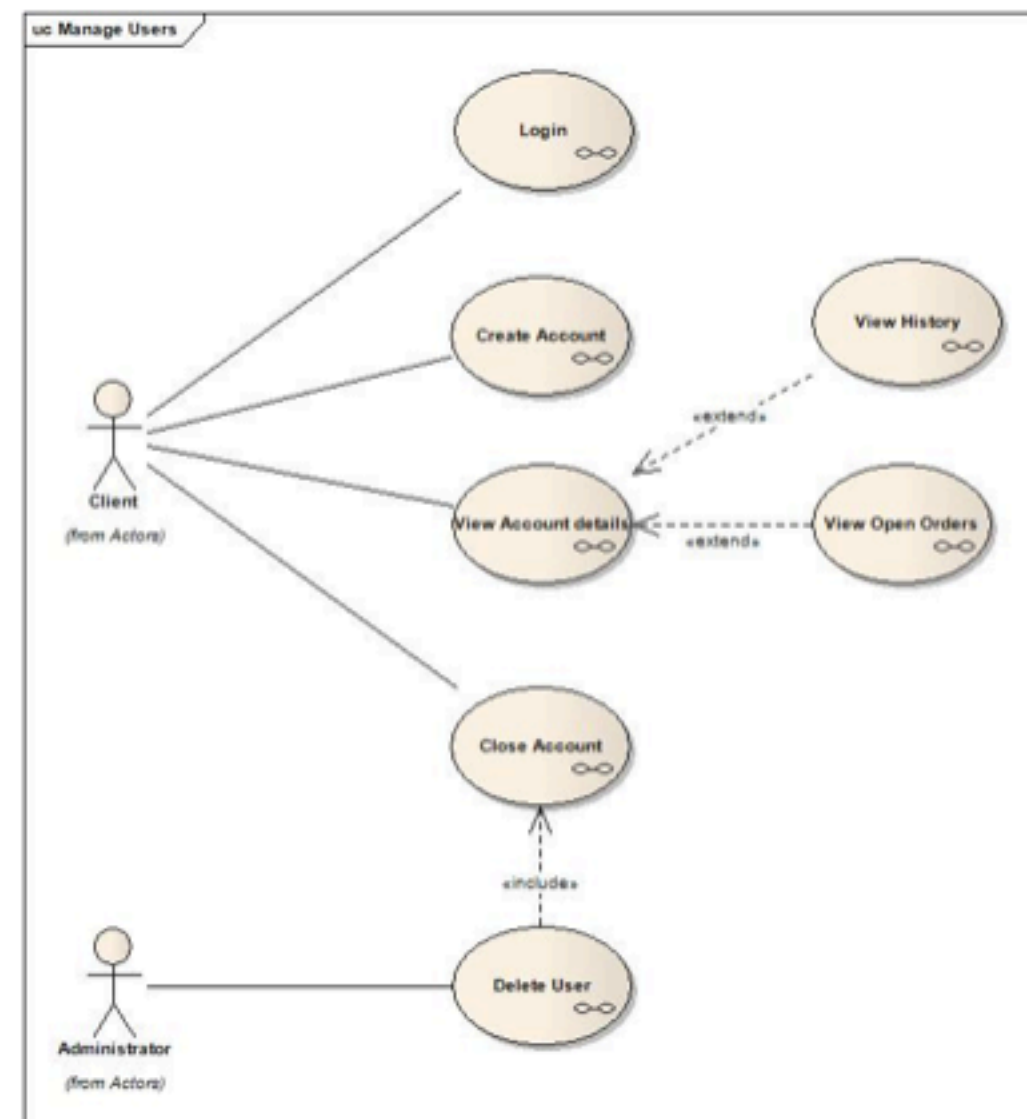
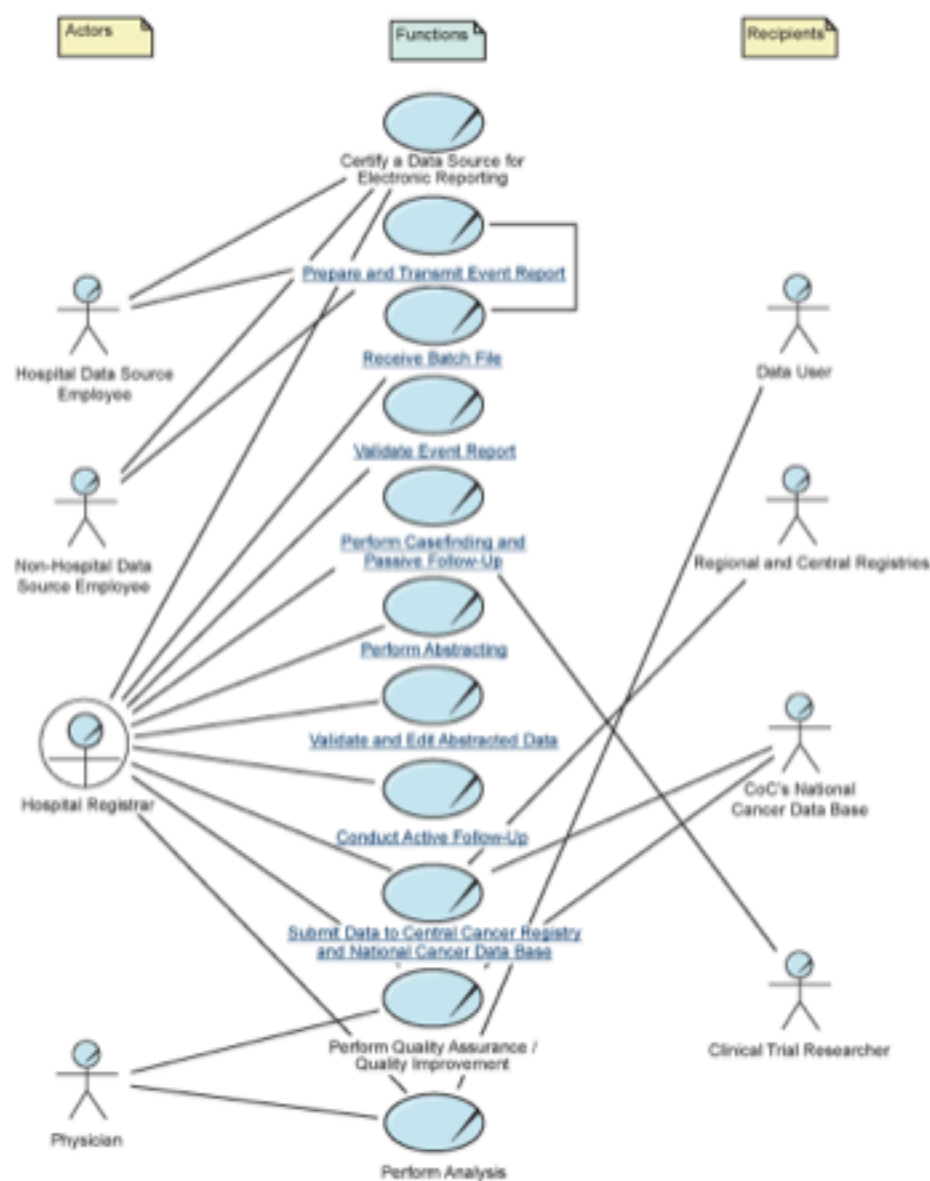


Диаграмма классов

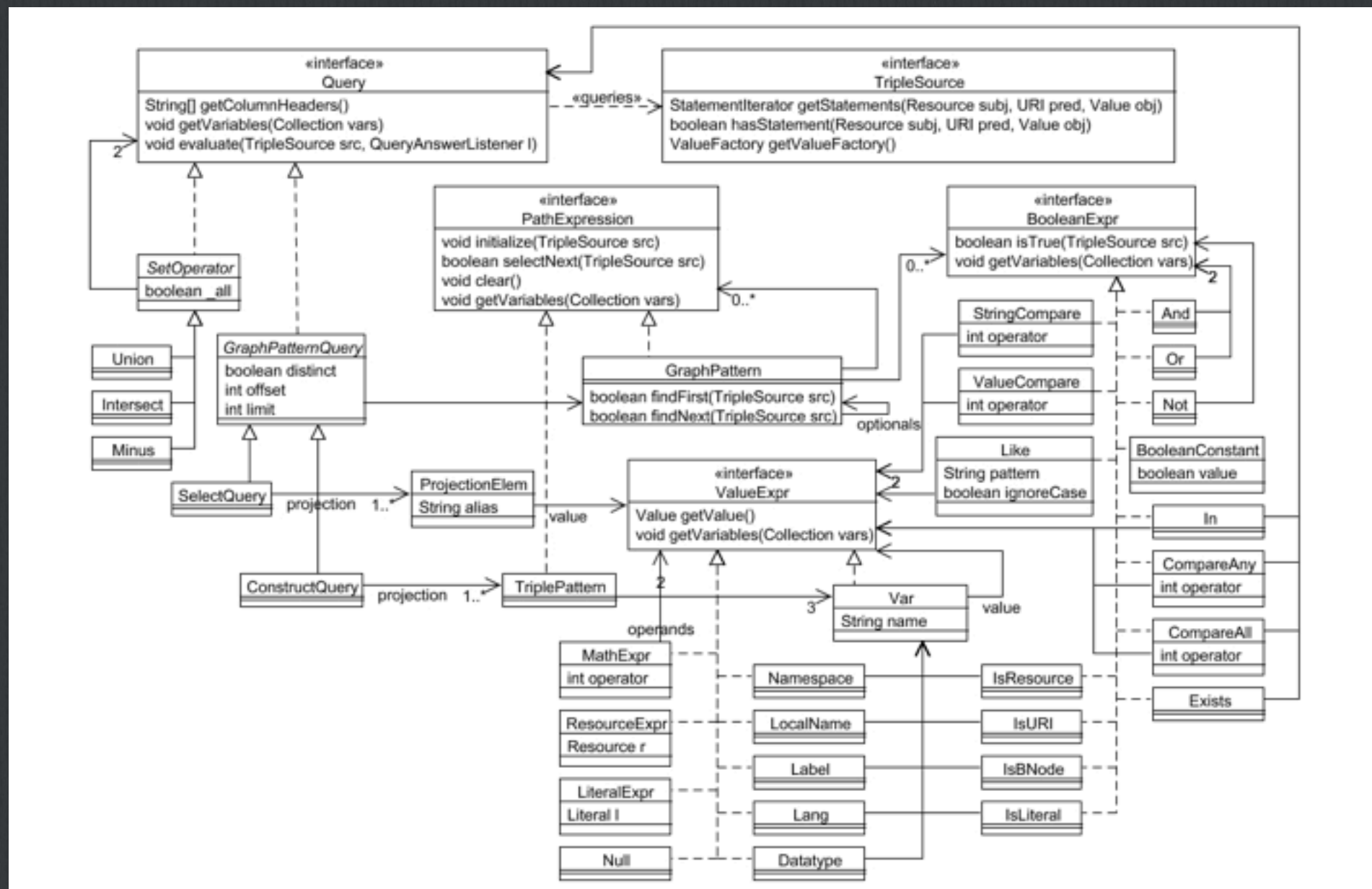


Диаграмма объектов

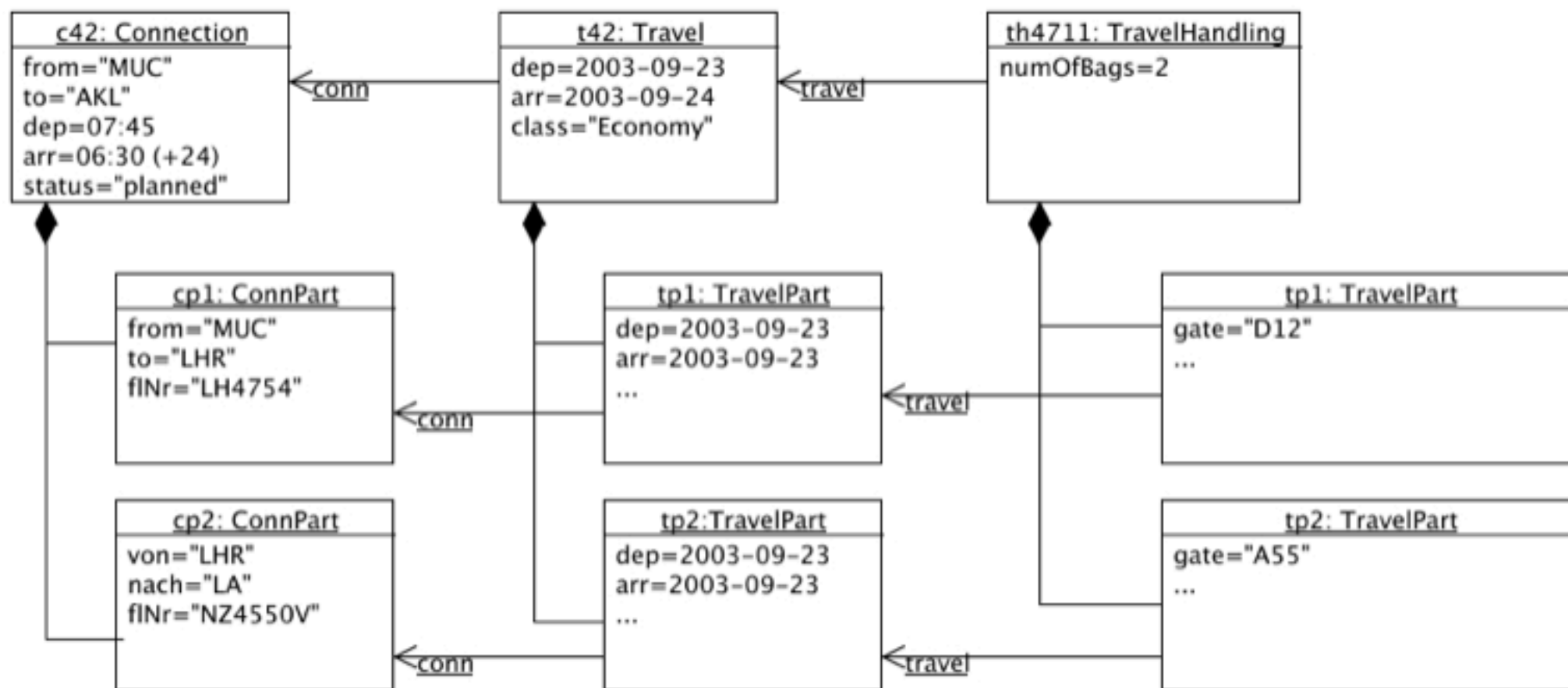


Диаграмма активностей

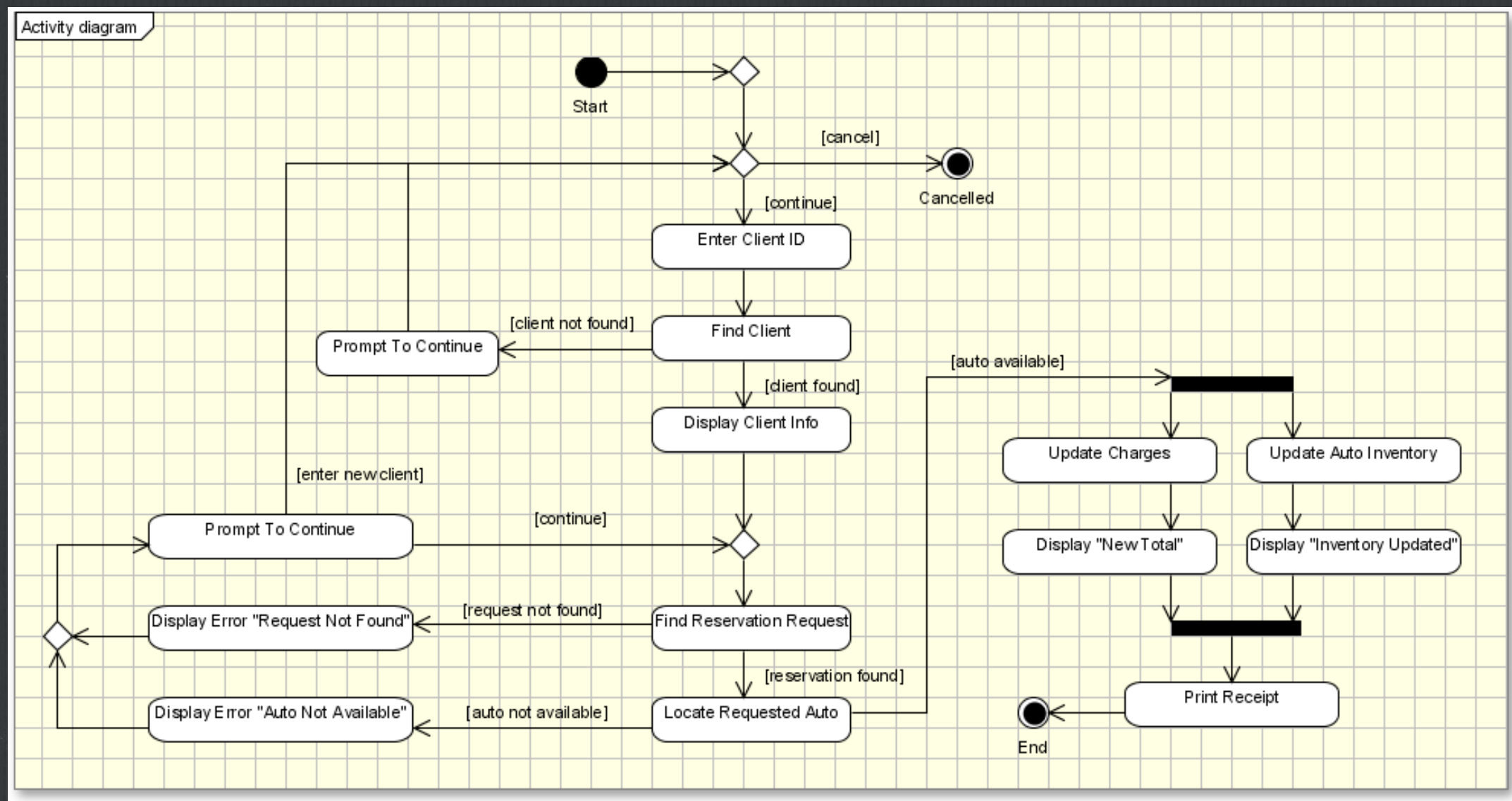


Диаграмма состояний

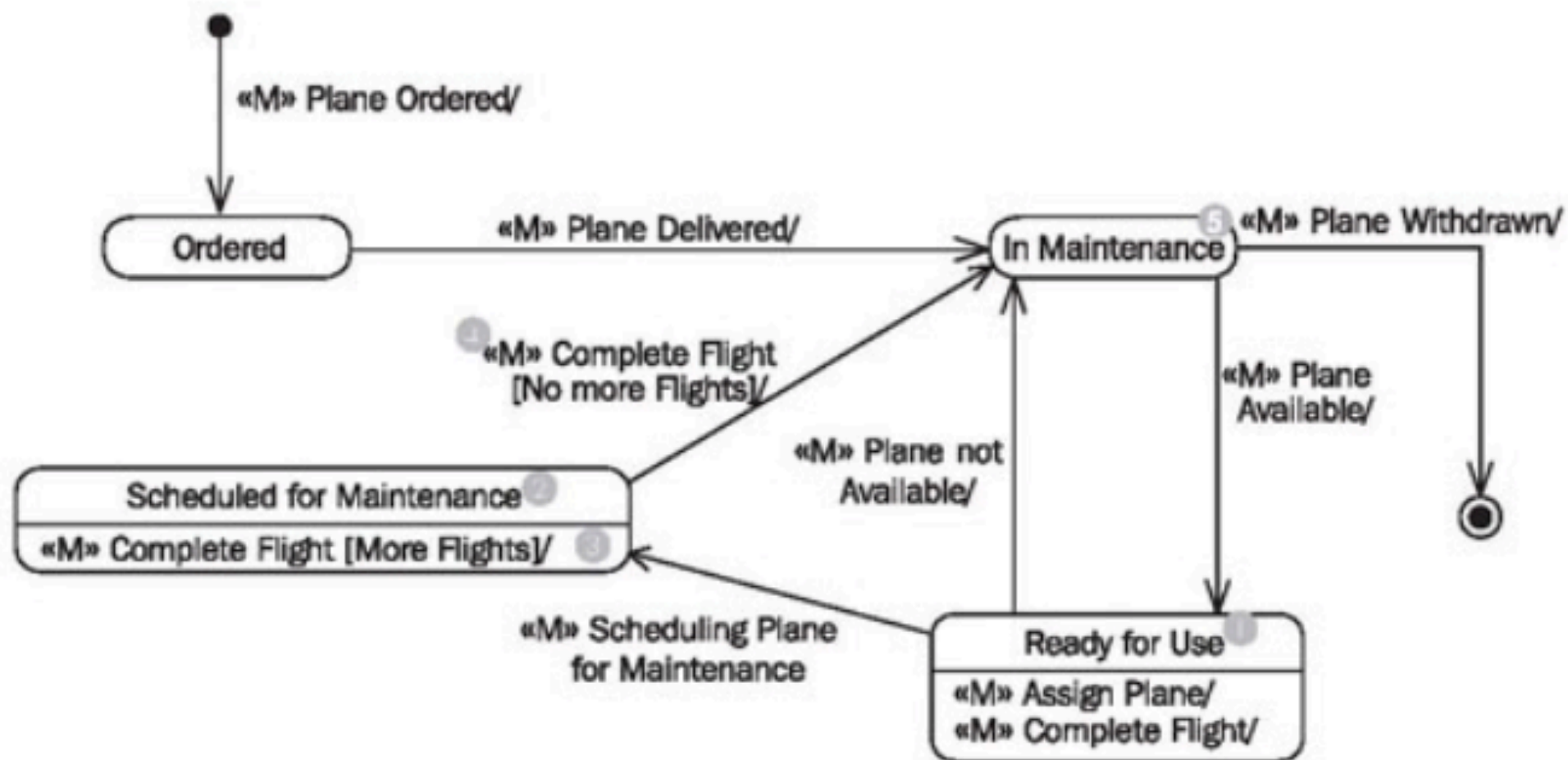


Диаграмма последовательности

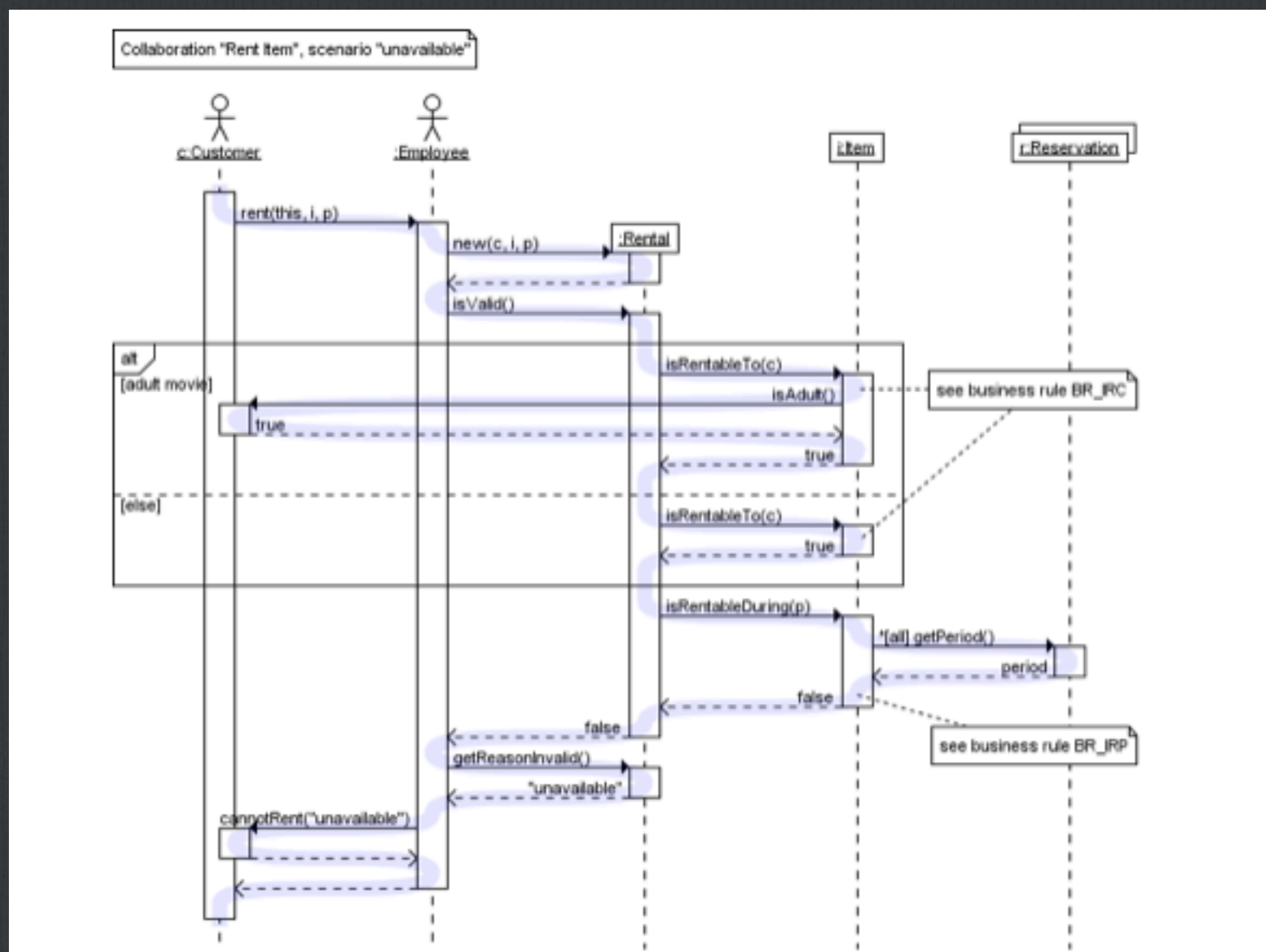


Диаграмма коопераций

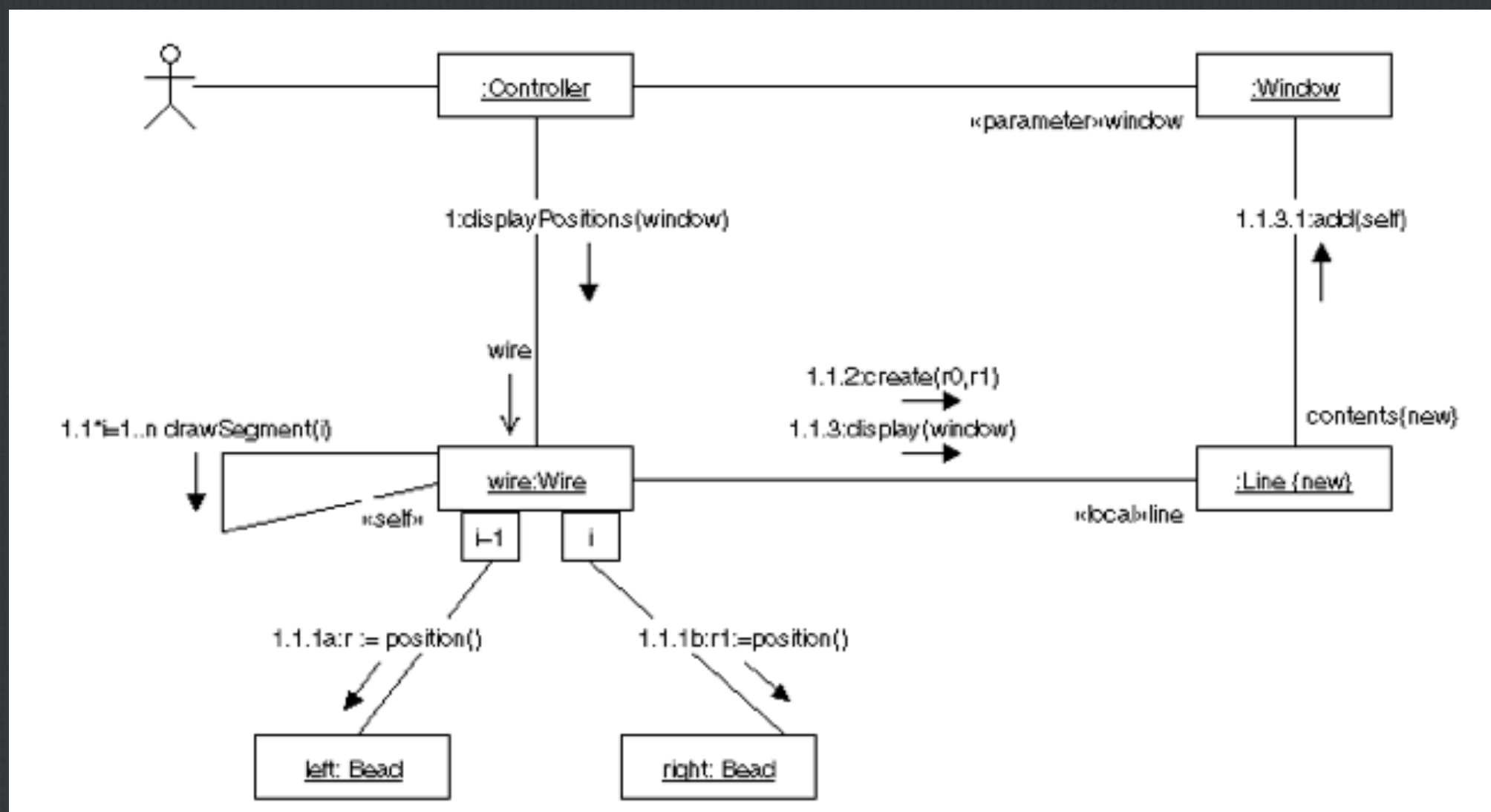


Диаграмма компонентов

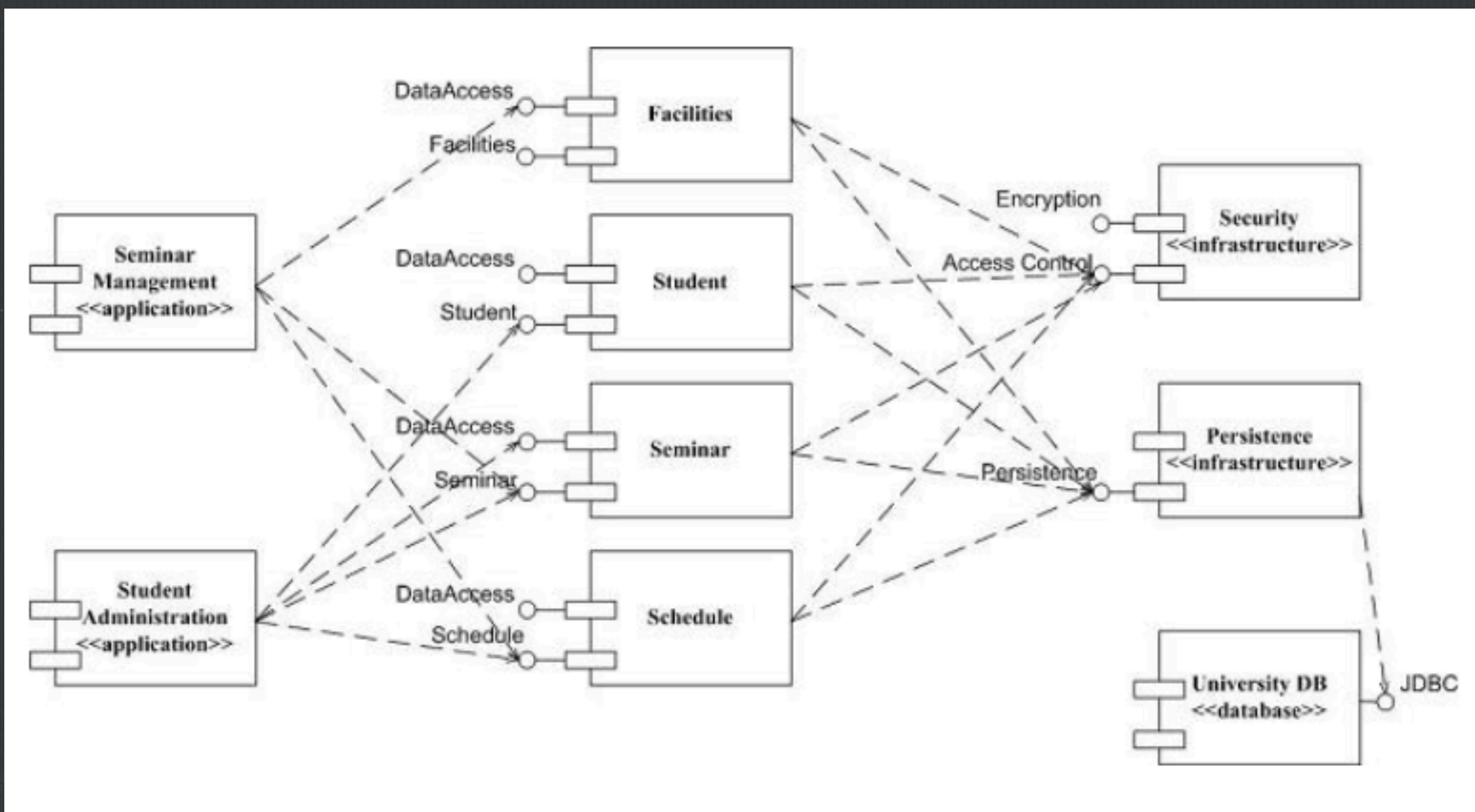
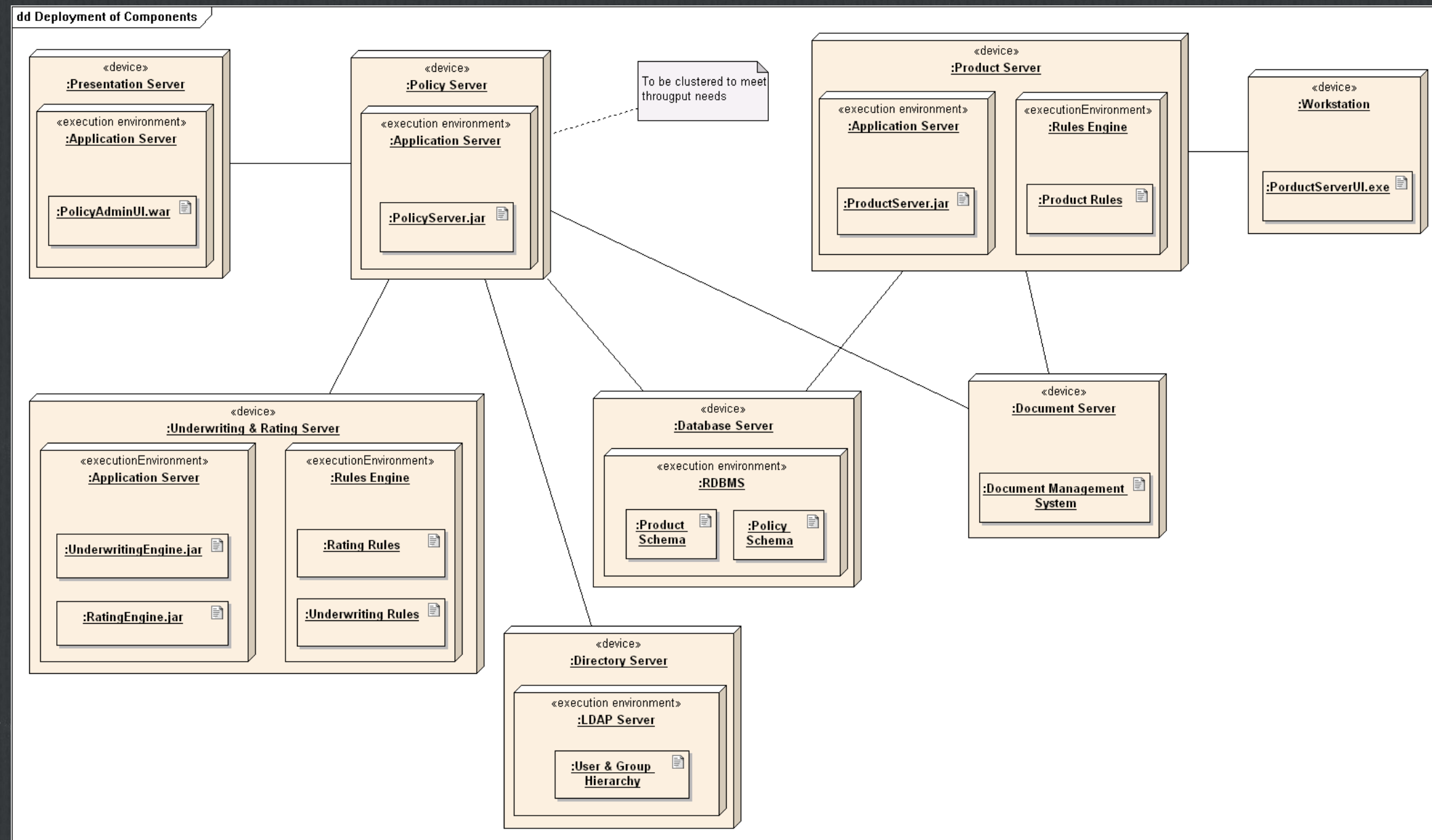


Диаграмма развёртывания



Правила языка UML

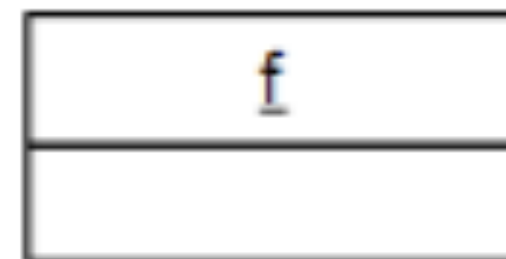
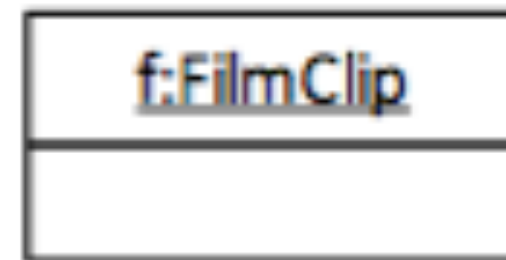
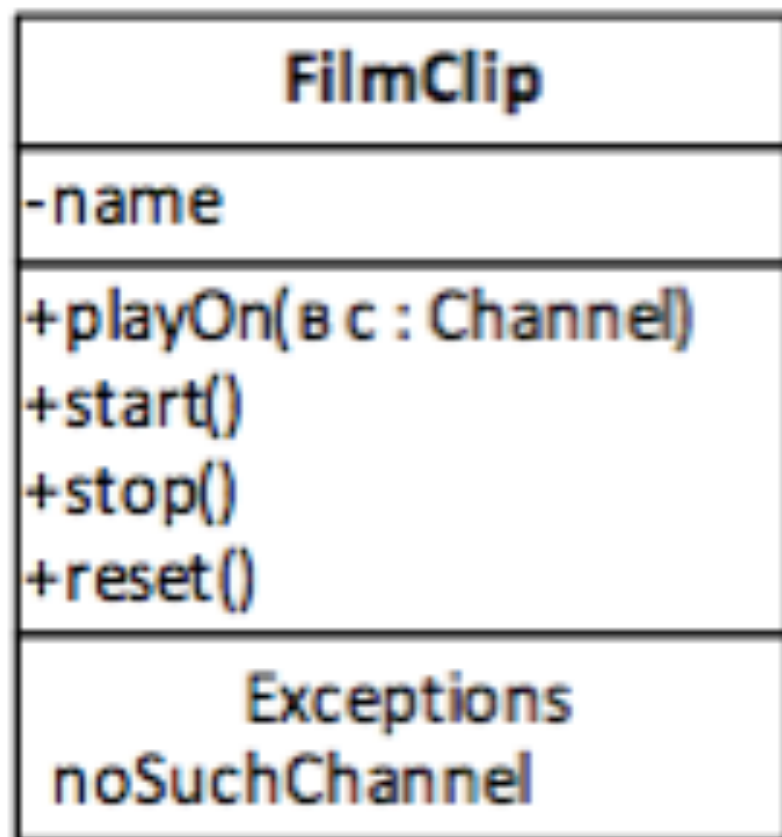
Семантические правила позволяют корректно определять:

- ☐ имена
- ☐ область действия
- ☐ ВИДИМОСТЬ
- ☐ целостность
- ☐ выполнение

Общие механизмы языка UML

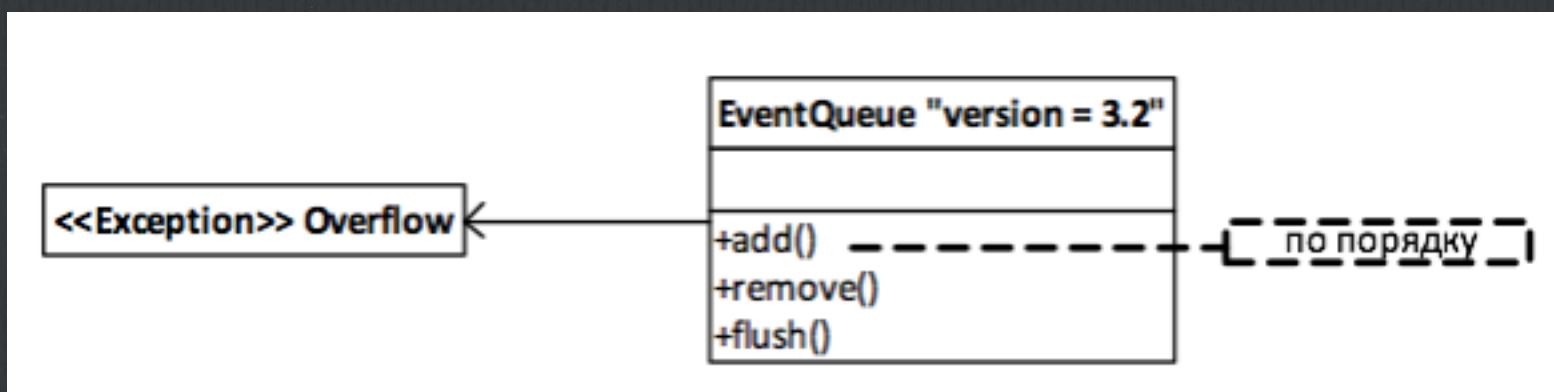
- ☐ Спецификации (specifications)
- ☐ Дополнения (adornments)
- ☐ Принятые деления (common divisions) – класс/
объект, интерфейс/реализация
- ☐ Механизмы расширения

Общие механизмы языка UML

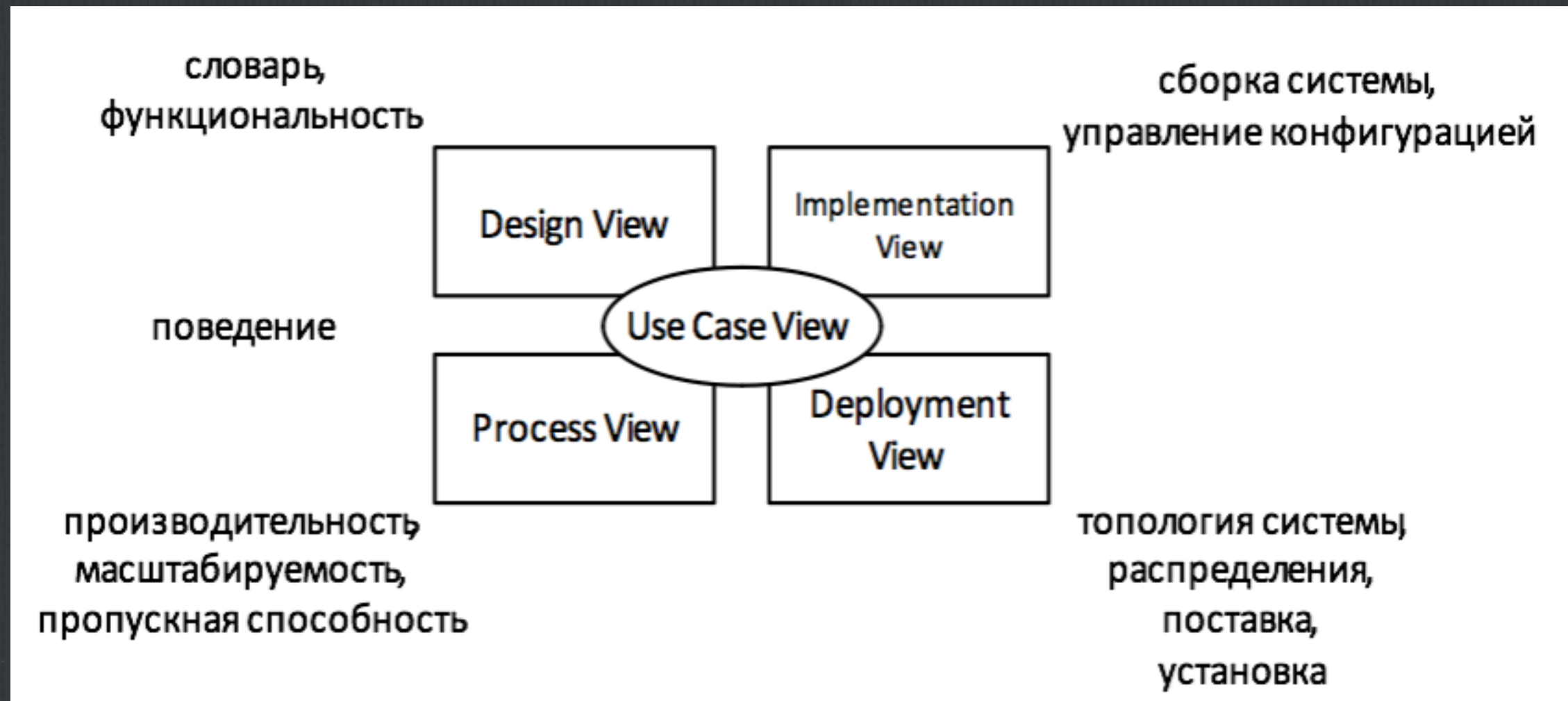


Механизмы расширения (extensibility mechanisms)

- ❑ Стереотипы (stereotypes)
- ❑ Помеченные значения (tagged values)
- ❑ Ограничения (constraints)



Архитектура программной системы



Rational Rose

